# CS-GY 6923: Lecture 12
# Autoencoders, and PCA

NYU Tandon School of Engineering, Akbar Rafiey

# Supervised vs. unsupervised learning

- **Supervised learning:** All input data examples come with targets/labels. What machines have been really good at for the past 8 years.
- **Unsupervised learning:** No input data examples come with targets/labels. Interesting problems to solve include clustering, anomaly detection, semantic embedding, etc.
- **Semi-supervised learning:** Some (typically very few) input data examples come with targets/labels. What human babies are really good at, and we have recently made machines a lot better at.

# Autoencoder

**Simple but clever idea:** If we have inputs $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^d$ but few or no targets $y_1, \ldots, y_n$, just make the inputs the targets.

- Let $f_{\boldsymbol{\theta}} : \mathbb{R}^d \to \mathbb{R}^d$ be our model.
- Let $L_{\boldsymbol{\theta}}$ be a loss function. E.g. squared loss: $L_{\boldsymbol{\theta}}(\mathbf{x}) = \|\mathbf{x} - f_{\boldsymbol{\theta}}(\mathbf{x})\|_2^2$.
- Train model: $\boldsymbol{\theta}^* = \min_{\boldsymbol{\theta}} \sum_{i=1}^n L_{\boldsymbol{\theta}}(\mathbf{x})$.

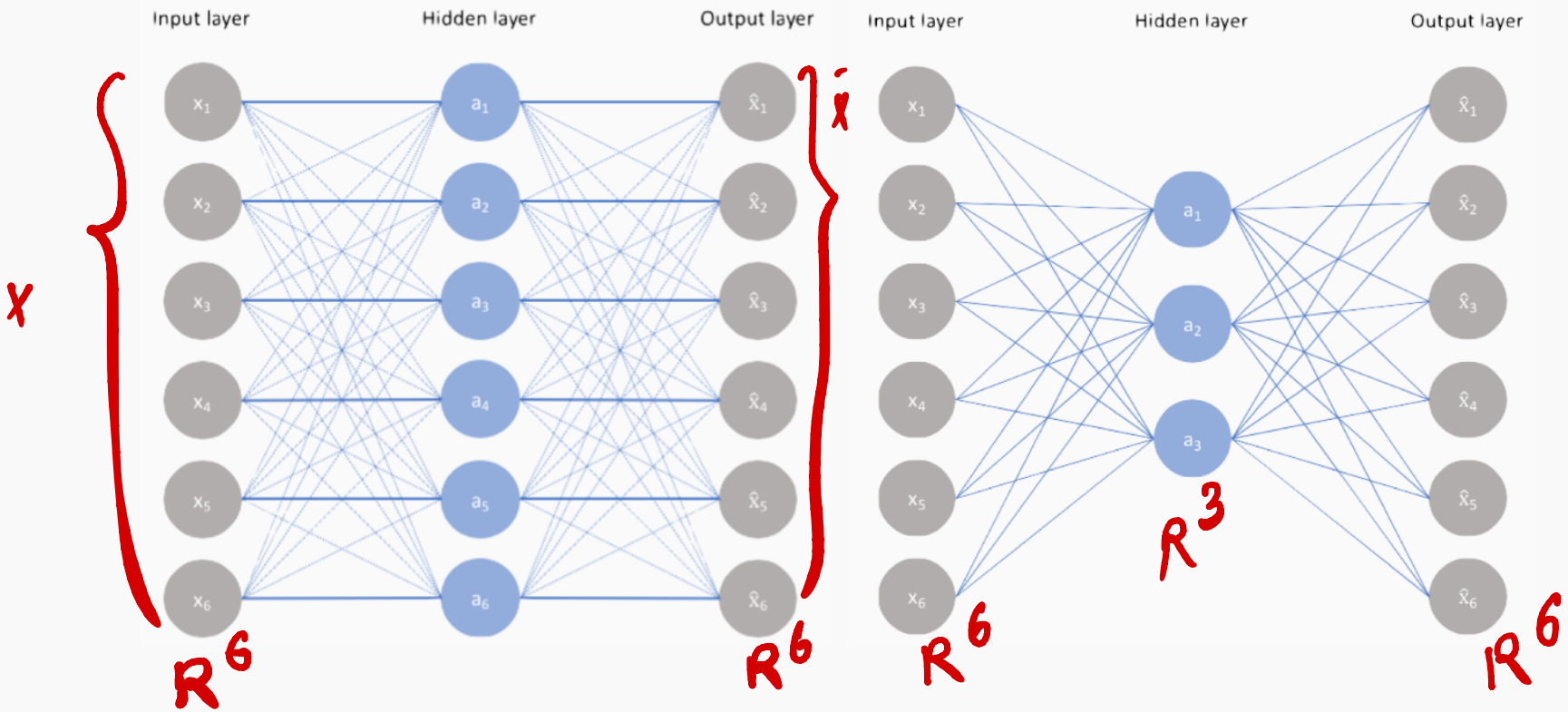
If $f_{\boldsymbol{\theta}}$ is a model that incorporates feature learning, then these features can be used for supervised tasks.

$f_{\boldsymbol{\theta}}$ is called an **autoencoder**. It maps input space to input space (e.g. images to images, french to french, PDE solutions to PDE solutions).

# Autoencoder

**Two examples of autoencoder architectures:**

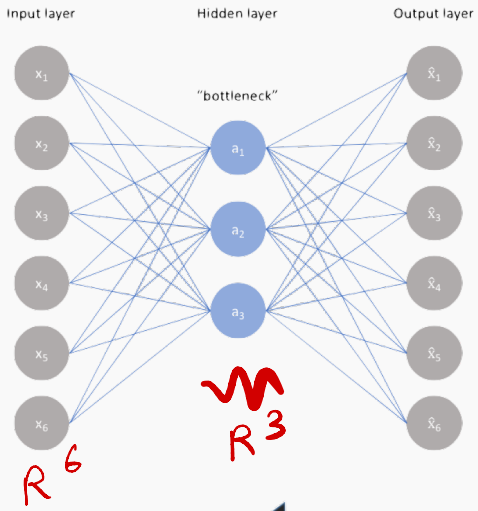
$\|x - \tilde{x}\|$ to be small

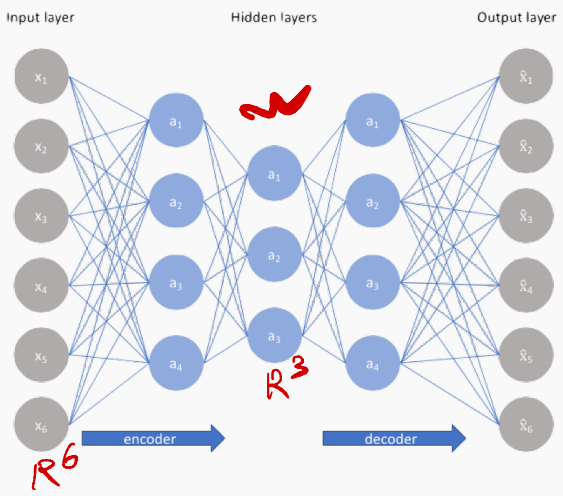


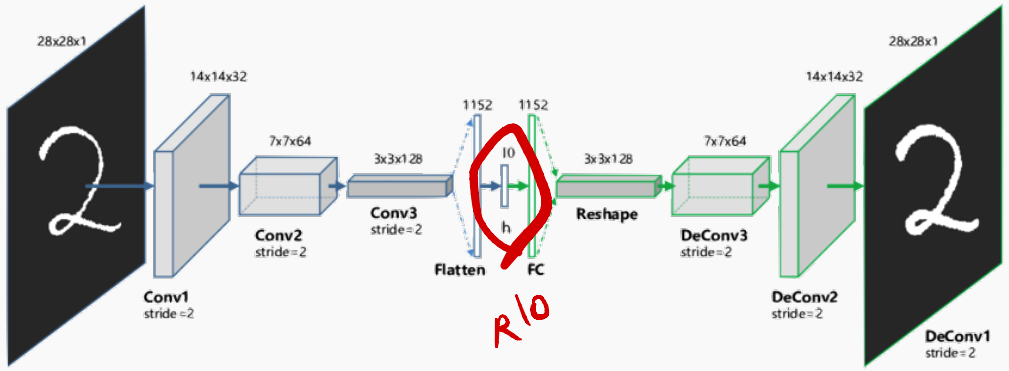

**Which would lead to better feature learning?**

# Autoencoder

**Important property of autoencoders**: no matter the architecture, there must always be a **bottleneck** with fewer parameters than the input. The bottleneck ensures information is "distilled" from low-level features to high-level features.



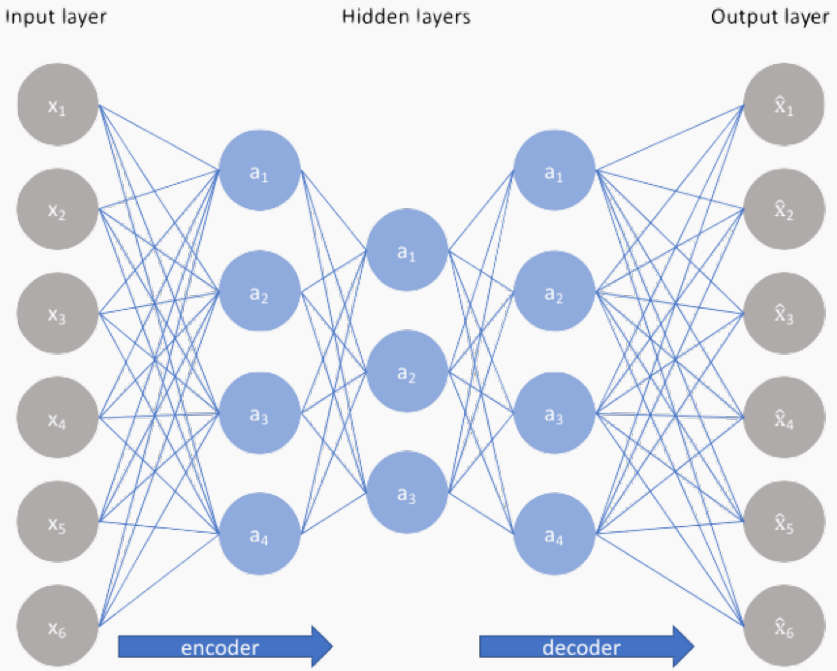
# Autoencoder

Separately name the mapping from input to bottleneck and from bottleneck to output.

**Encoder:** $e : \mathbb{R}^d \to \mathbb{R}^k$ **Decoder:** $d : \mathbb{R}^k \to \mathbb{R}^d$

$k < d$

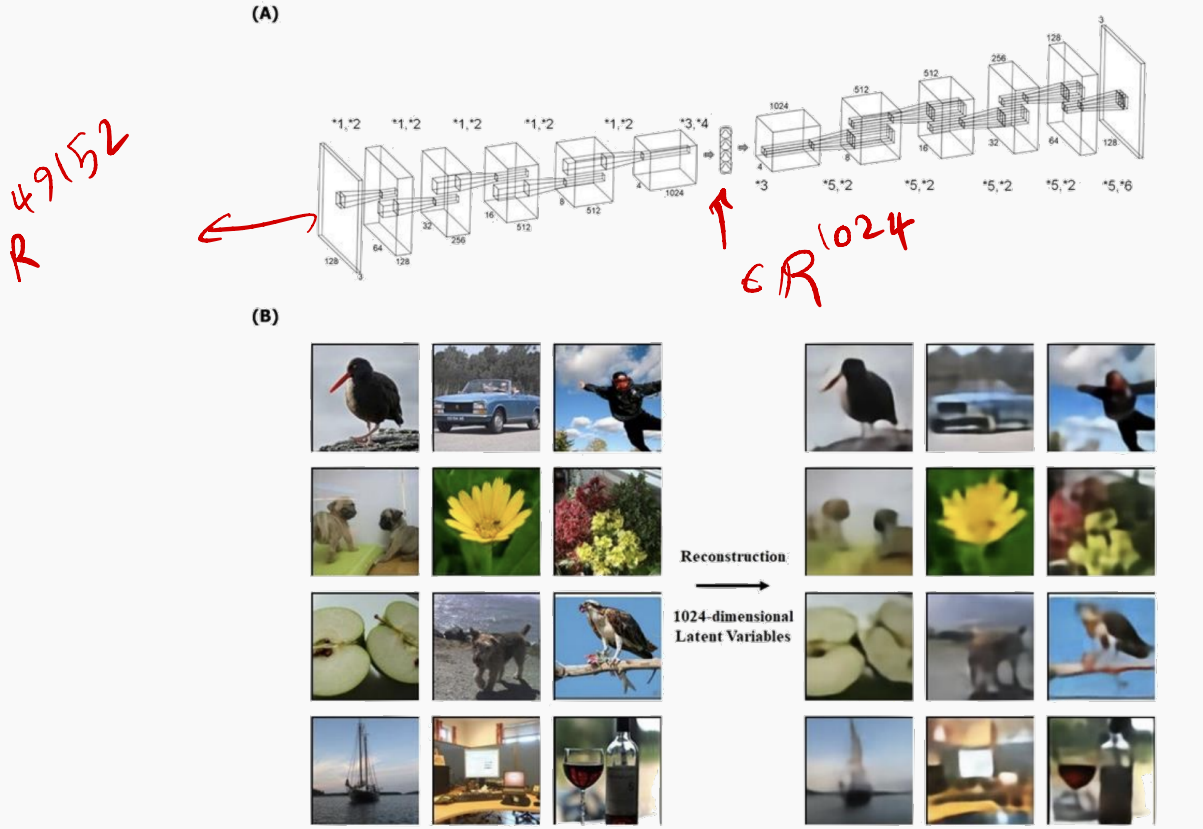
$f(\mathbf{x}) = d(e(x))$




Often symmetric, but does not have to be.

# Autoencoder reconstruction

**Example image reconstructions from autoencoder:**




https://www.biorxiv.org/content/10.1101/214247v1.full.pdf

**Input parameters:** $d = 49152$.

**Bottleneck "latent" parameters:** $k = 1024$.

# Autoencoders for feature extraction

The best autoencoders do not work as well as supervised methods for feature extraction, but they require no labeled data.[1]

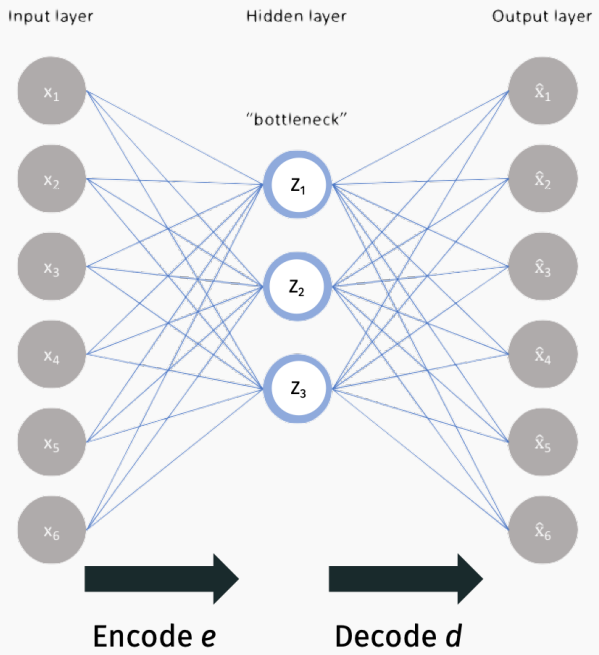
There are a lot of cool applications of autoencoders beyond feature learning!

- Learned data compression.
- Denoising and in-painting.
- Data/image synthesis.

[1] Recent progress on **self-supervised** learning achieves the best of both worlds – state-of-the-art feature learning with no labeled data.

# Autoencoders for data compression

Due to their bottleneck design, autoencoders perform **dimensionality reduction** and thus data compression.



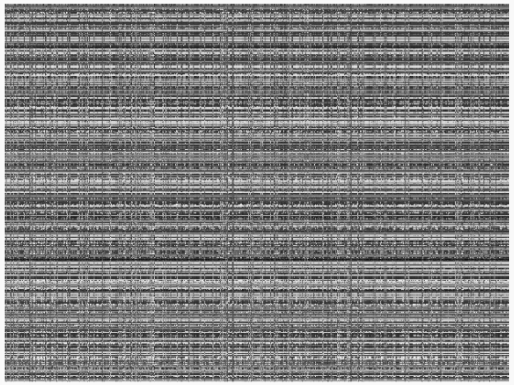

Given input $\mathbf{x}$, we can completely recover $f(\mathbf{x})$ from $\mathbf{z} = e(\mathbf{x})$. $\mathbf{z}$ typically has many fewer dimensions than $\mathbf{x}$ and for a typical image $f(\mathbf{x})$ will closely approximate $\mathbf{x}$.

# Autoencoders for image compression

The best lossy compression algorithms are tailor made for specific types of data:

- JPEG 2000 for images
- MP3 for digital audio.
- MPEG-4 for video.

All of these algorithms take advantage of specific structure in these data sets. E.g. JPEG assumes images are locally "smooth".

# Autoencoders for image compression

With enough input data, autoencoders can be trained to find this structure on their own.

**Proposed method**, 5908 bytes (0.167 bit/px), PSNR: luma 23.38 dB/chroma 31.86 dB, MS-SSIM: 0.9219

**Proposed method**, 6021 bytes (0.170 bit/px), PSNR: 24.12 dB, MS-SSIM: 0.9292

**JPEG 2000**, 5908 bytes (0.167 bit/px), PSNR: luma 23.24 dB/chroma 31.04 dB, MS-SSIM: 0.8803

**JPEG 2000**, 6037 bytes (0.171 bit/px), PSNR: 23.47 dB, MS-SSIM: 0.9036

"End-to-end optimized image compression", Ballé, Laparra, Simoncelli

Need to be careful about how you choose loss function, design the network, etc. but can lead to much better image compression than "hand-tuned" algorithms like JPEG.

# Autoencoders for image correction

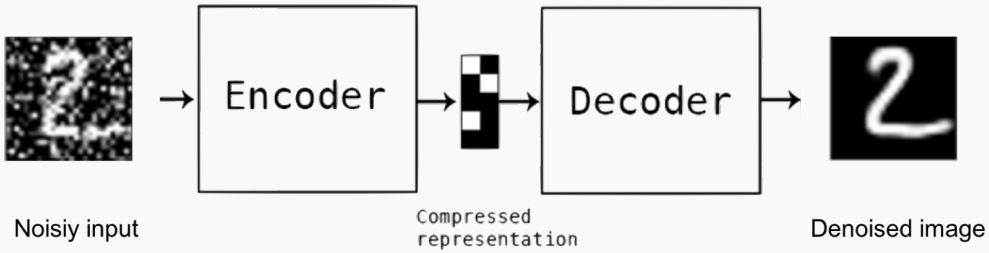



Image denoising

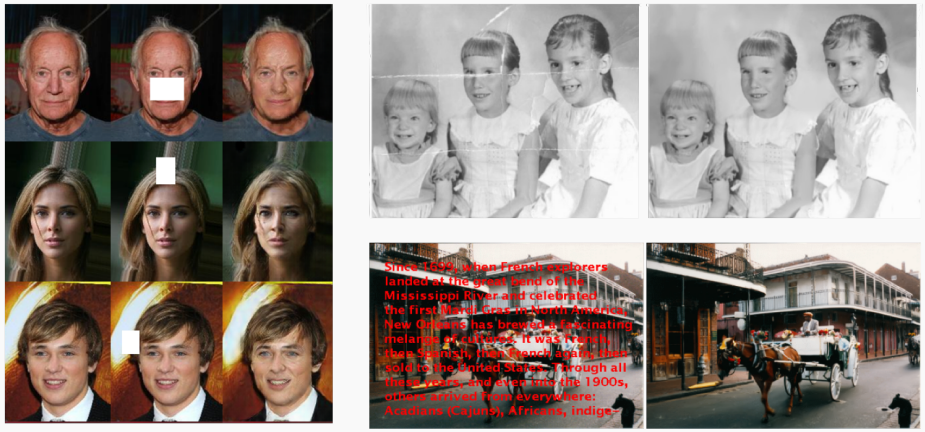
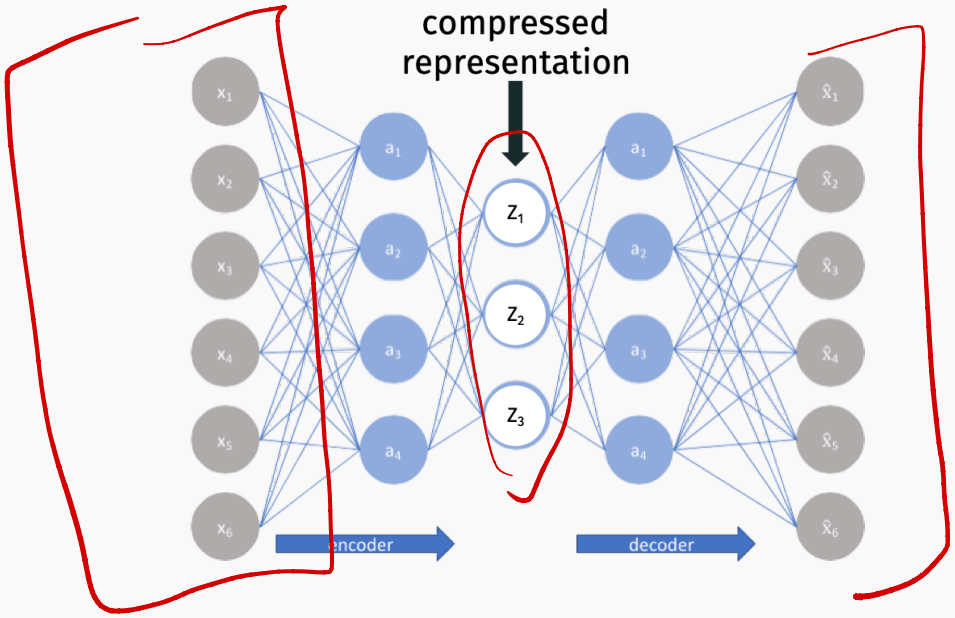


Image inpainting

Train autoencoder on uncorrupted images (unsupervised). Pass corrupted image $\mathbf{x}$ through autoencoder and return $f(\mathbf{x})$ as repaired result.

# Autoencoders learn compressed representations

## Why does this work?



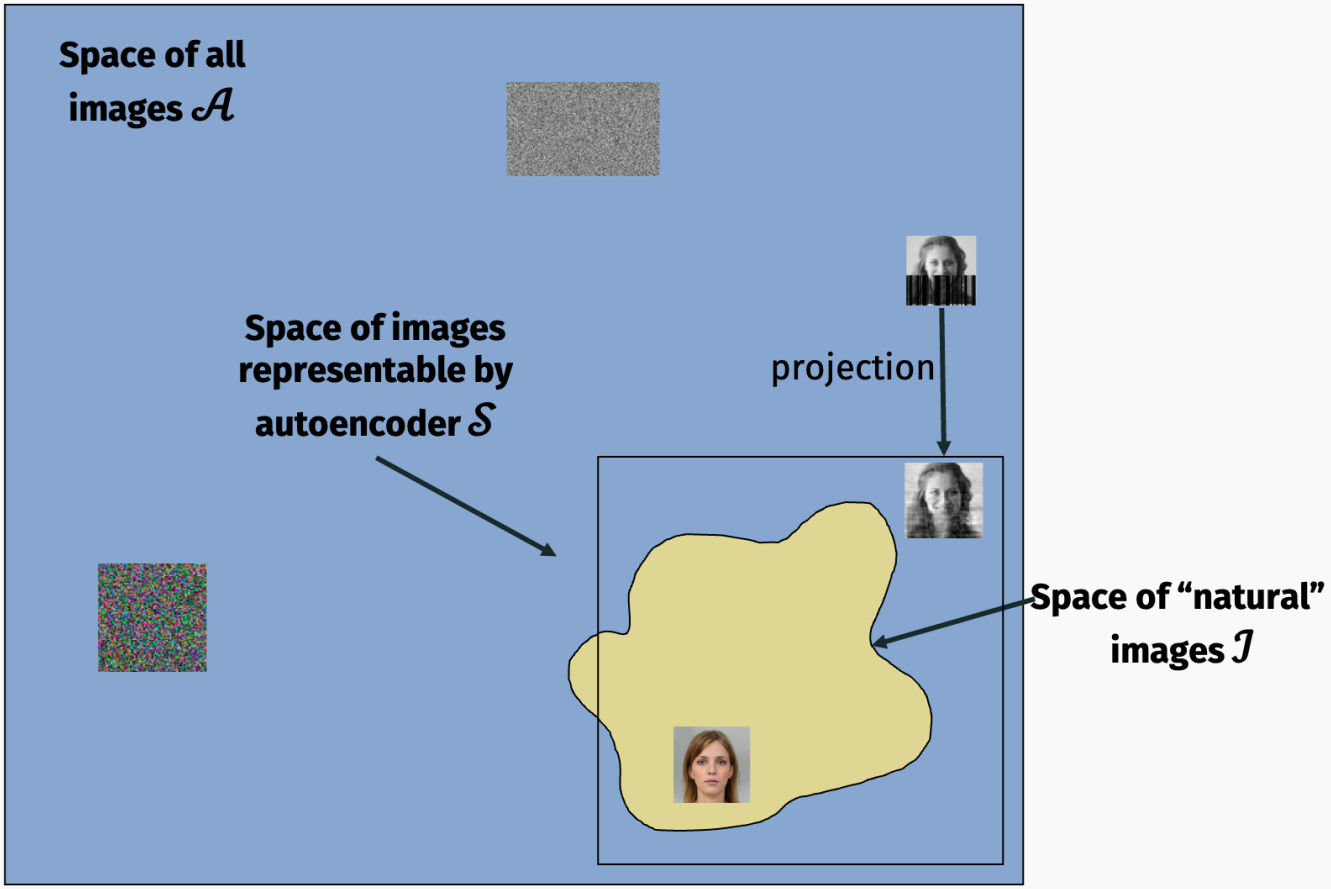

Consider $128 \times 128 \times 3$ images with pixels values in $0, 1 \ldots, 255$.
How many possible images are there?

$(256)^{128 \times 128 \times 3}$

If $\mathbf{z}$ holds $k$ values in $0, .1, .2, \ldots, 1$, how many unique images $\mathbf{w}$ can be output by the autoencoder function $f$?

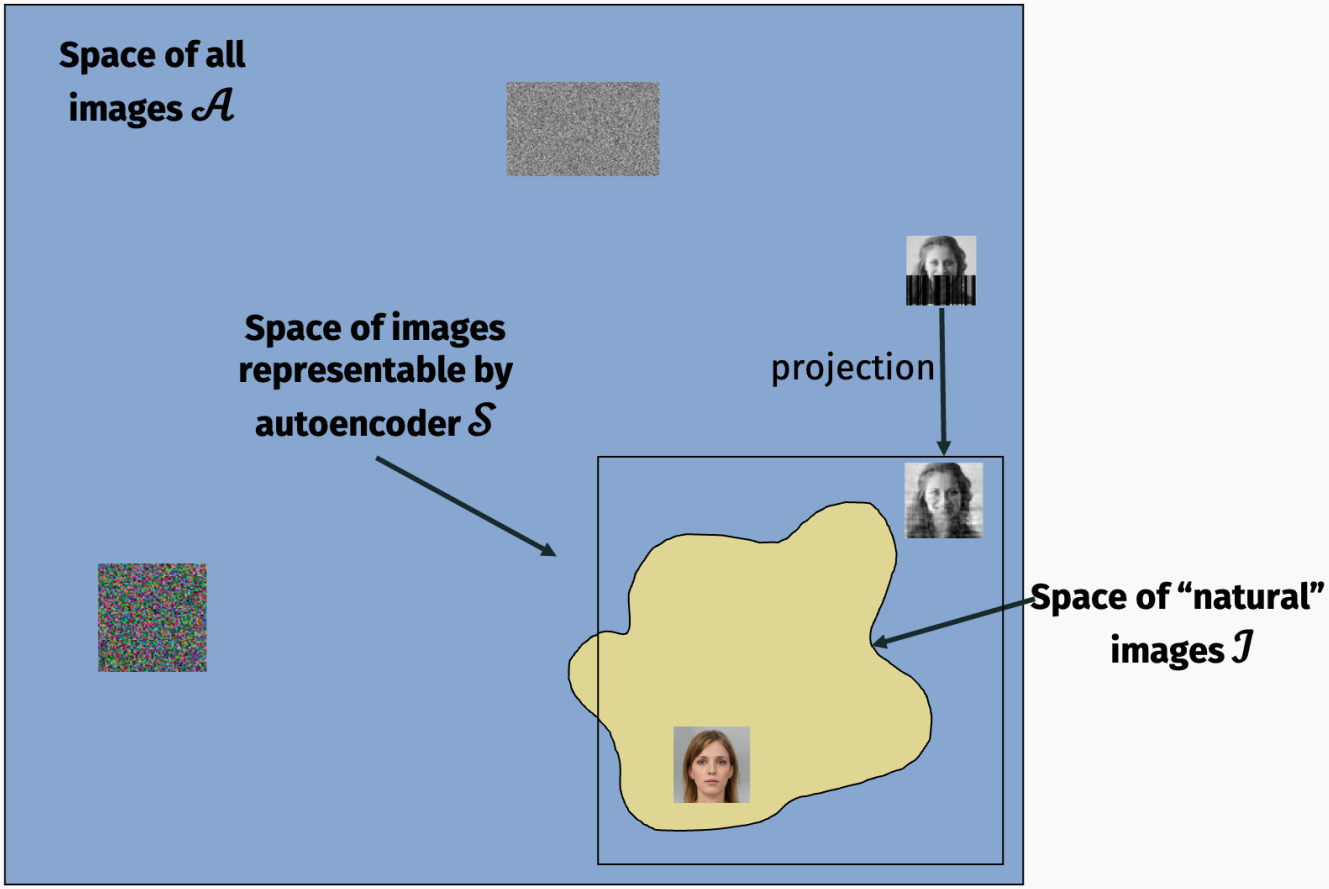
$k^3$

# Autoencoders learn compressed representations




For a good (accurate, small bottleneck) autoencoder, $\mathcal{S}$ will closely approximate $\mathcal{I}$. Both will be much smaller than $\mathcal{A}$.

# Autoencoders learn compressed representations

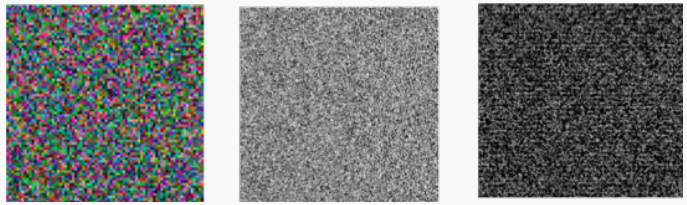



$f(\mathbf{x}) = d(e(\mathbf{x}))$ projects an image $\mathbf{x}$ closer to the space of natural images.

# Autoencoders for data generation

Suppose we want to generate a random natural image. How might we do that?

- **Option 1**: Draw each pixel value in $\mathbf{x}$ uniformly at random. Draws a random image from $\mathcal{A}$.

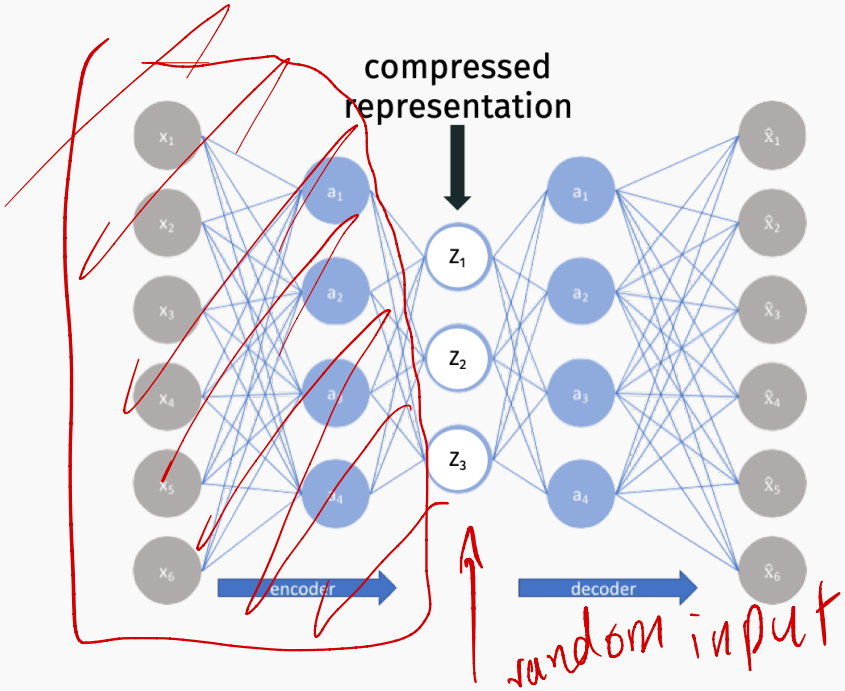


- **Option 2**: Draw $\mathbf{x}$ randomly from $\mathcal{S}$, the space of images representable by the autoencoder.

How do we randomly select an image from $\mathcal{S}$?

# Autoencoders for data generation

How do we randomly select an image $\mathbf{x}$ from $\mathcal{S}$?

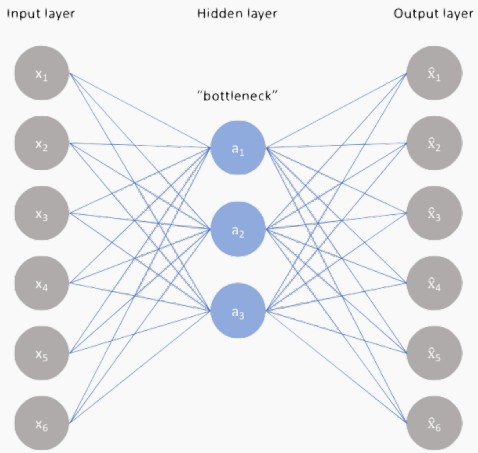


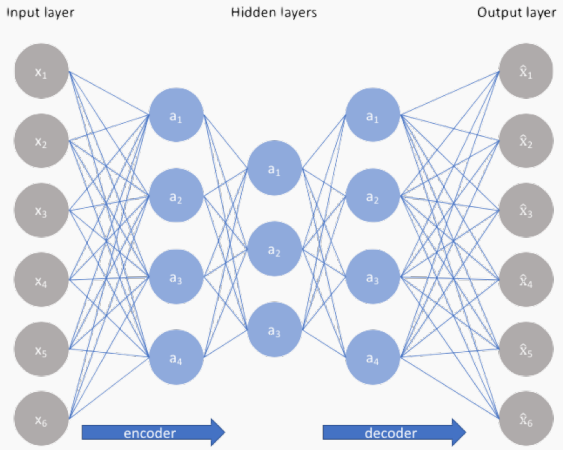

Randomly select code $\mathbf{z}$, then set $\mathbf{x} = d(\mathbf{z})$.[2]

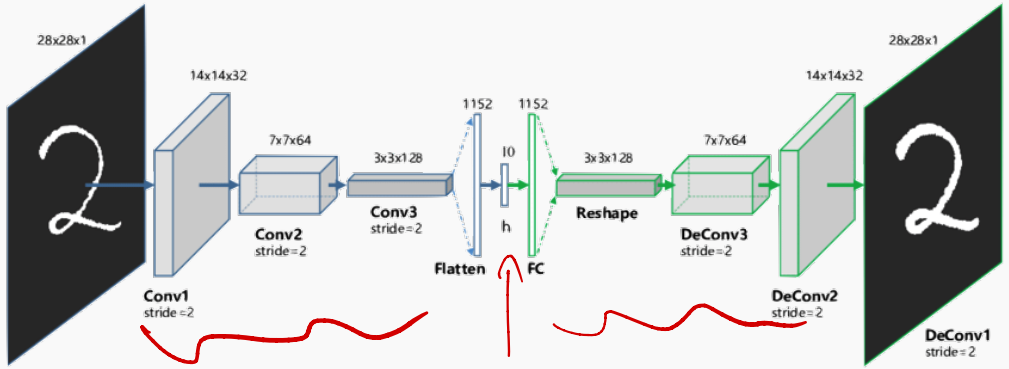
[2] Lots of details to think about here. In reality, people use "variational autoencoders" (VAEs), which are a natural modification of AEs.

# Autoencoder

Goal of autoencoder models is to map input data to a close approximation of the original that takes less space to represent.



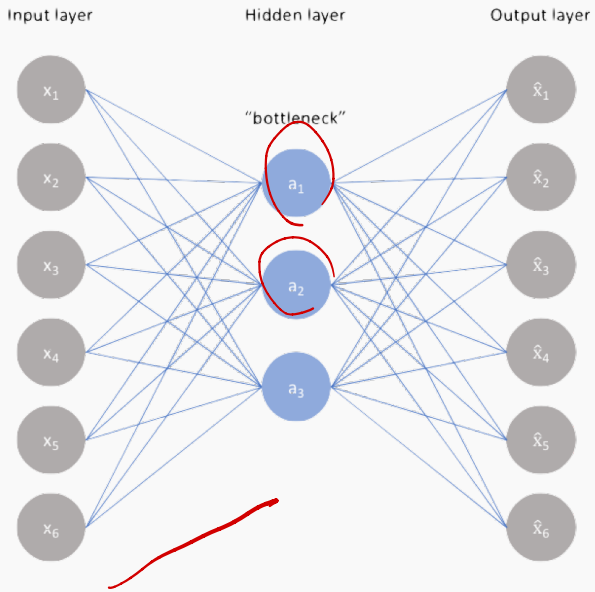



# Principal Component Analysis

# Principal Component Analysis

PCA is the "linear regression" of autoencoders:

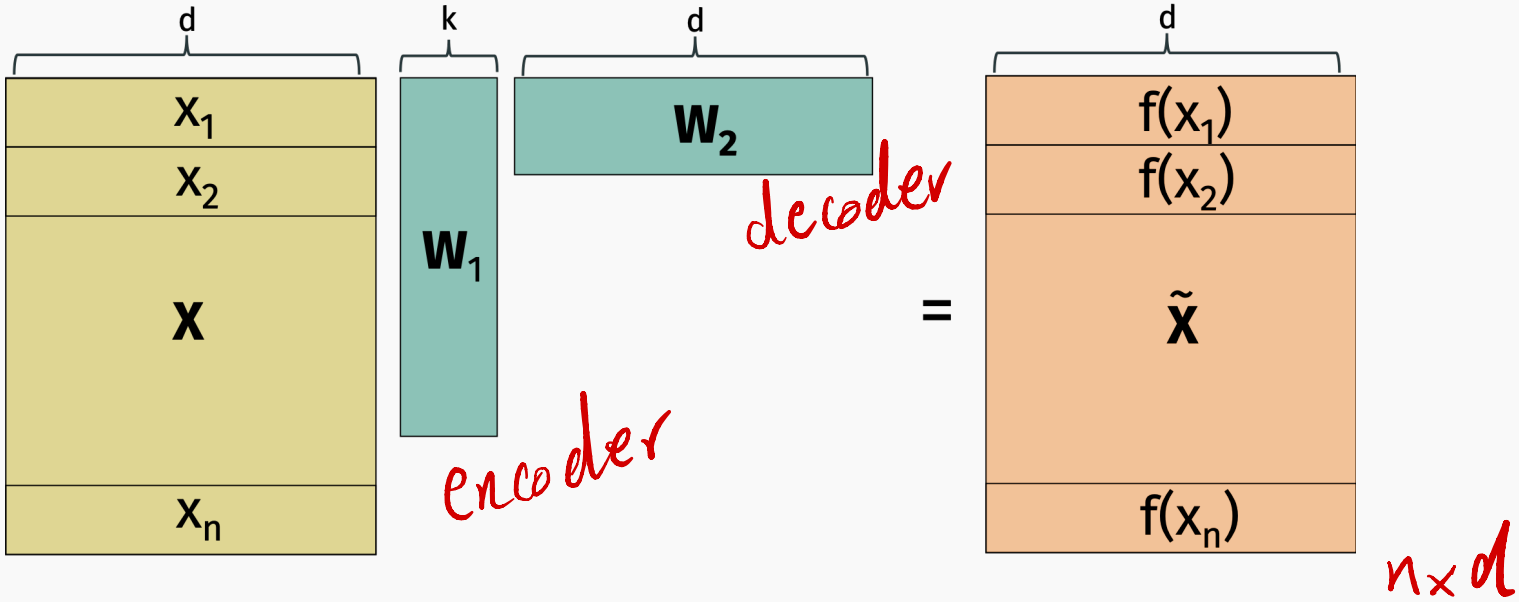



- Simplest possible model. One layer, no non-linearities.
  - $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{W}_1\mathbf{W}_2$ where $\mathbf{X} \in \mathbb{R}^{n \times d}, \mathbf{W}_1 \in \mathbb{R}^{d \times k}, \mathbf{W}_2 \in \mathbb{R}^{k \times d}$.
  - Want to minimize $\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{X} - \mathbf{X}\mathbf{W}_1\mathbf{W}_2\|_F^2$.

data matrix

# Principal Component Analysis (PCA)

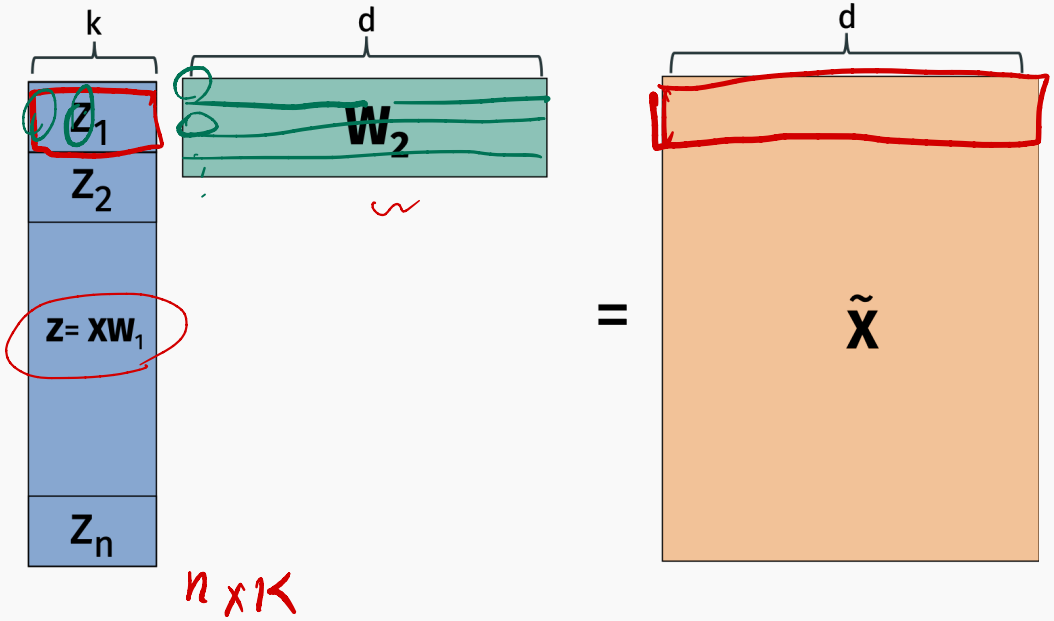
Given training data set $\mathbf{x}_1, \ldots, \mathbf{x}_n$, let $\mathbf{X}$ denote our data matrix.
Let $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{W}_1\mathbf{W}_2$.




$x_1 \approx f(x_1)$
$x_2 \approx f(x_2)$ ...

# Low-rank approximation

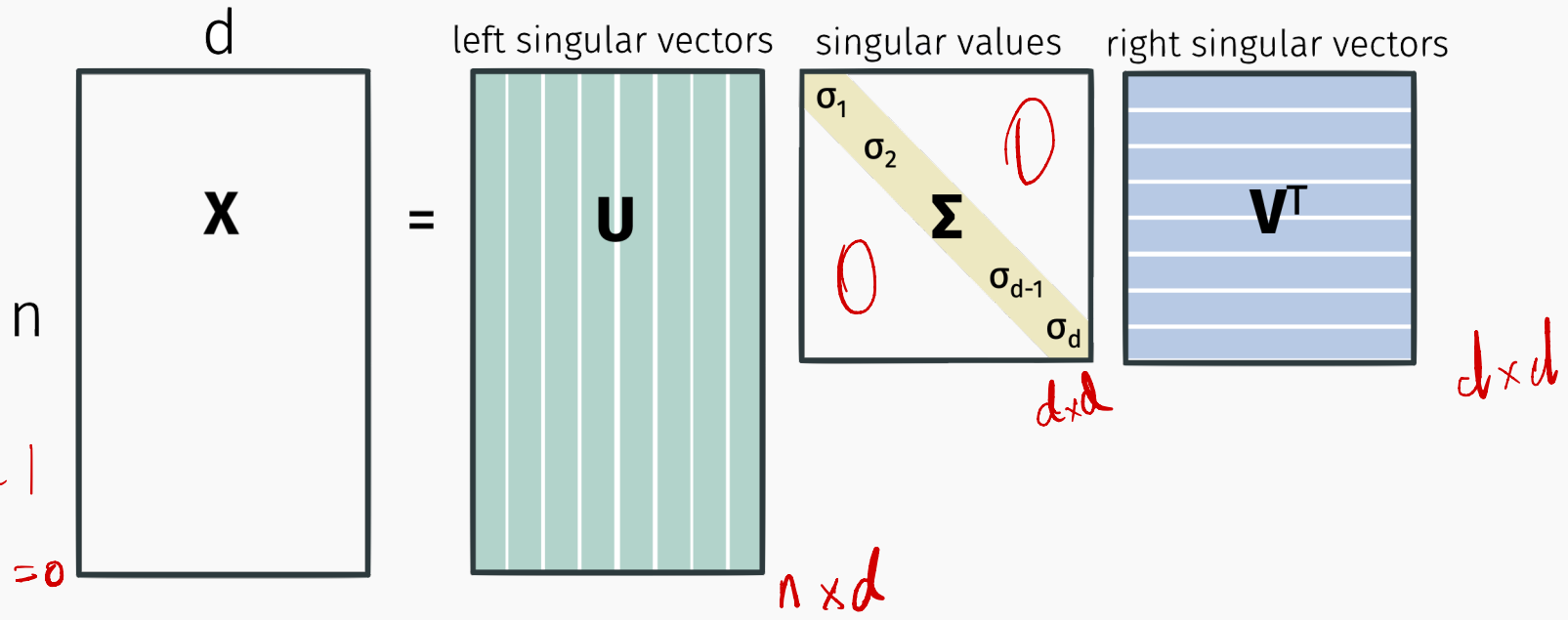
- $\tilde{\mathbf{X}}$ is a **low-rank matrix**. It only has rank (at most) $k$ for $k \ll d$.
- Finding best $\mathbf{W}_1, \mathbf{W}_2$ : equivalent to low-rank approximation. Can be efficiently and provable optimized using the SVD.




every row in $\tilde{X}$ is a linear combination of the k rows in $W_2$.

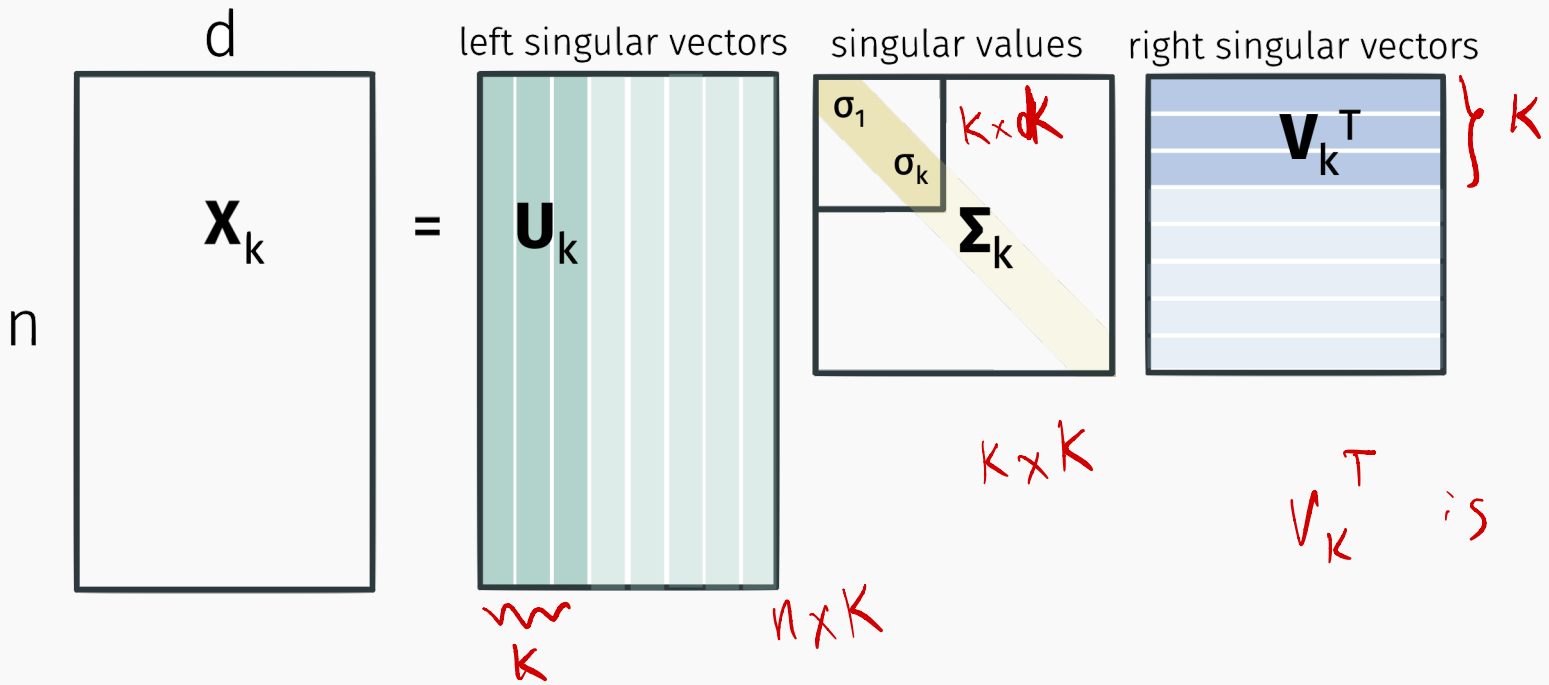
# Singular Value Decomposition

Any matrix $\mathbf{X}$ can be written:




Where $\mathbf{U}^T\mathbf{U} = \mathbf{I}$, $\mathbf{V}^T\mathbf{V} = \mathbf{I}$, and $\sigma_1 \geq \sigma_2 \geq \ldots \sigma_d \geq 0$. I.e. $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices. Can be computed in $O(nd^2)$ time (faster with approximation algos).

# Partial Singular Value Decomposition

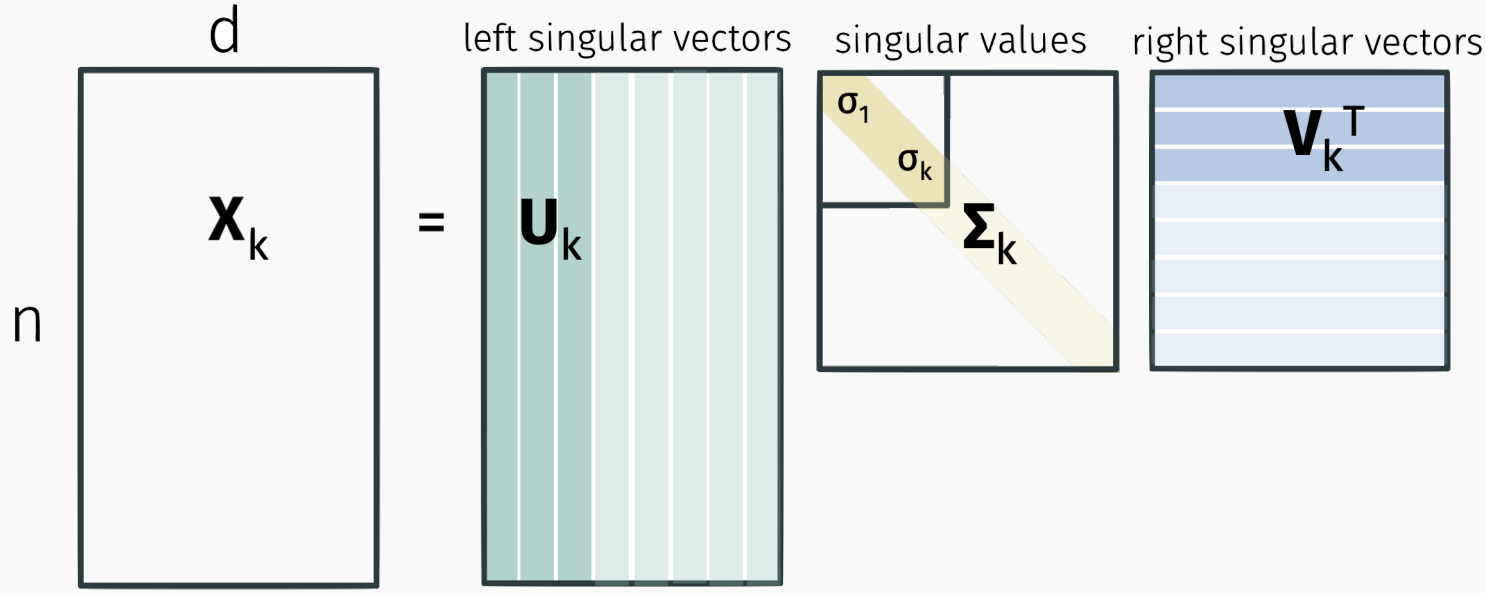



Can be computed in roughly $O(ndk)$ time.

$O(nd^2)$

# Singular value decomposition

Can read off optimal low-rank approximations from the SVD:




**Eckart–Young–Mirsky Theorem:** For any $k \leq d$,
$\mathbf{X}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T$ is the optimal $k$ rank approximation to $\mathbf{X}$:

$$\mathbf{X}_k = \underset{\tilde{\mathbf{X}} \text{ with rank} \leq k}{\arg\min} \|\mathbf{X} - \tilde{\mathbf{X}}\|_F^2.$$

# Singular value decomposition

**Claim:** $\mathbf{X}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T = \mathbf{X}\mathbf{V}_k\mathbf{V}_k^T$.

$n \times d$ $d \times k$ $k \times d$

our goal $\longrightarrow U_k \Sigma_k V_k^T = U \Sigma V^T V_k V_k^T$

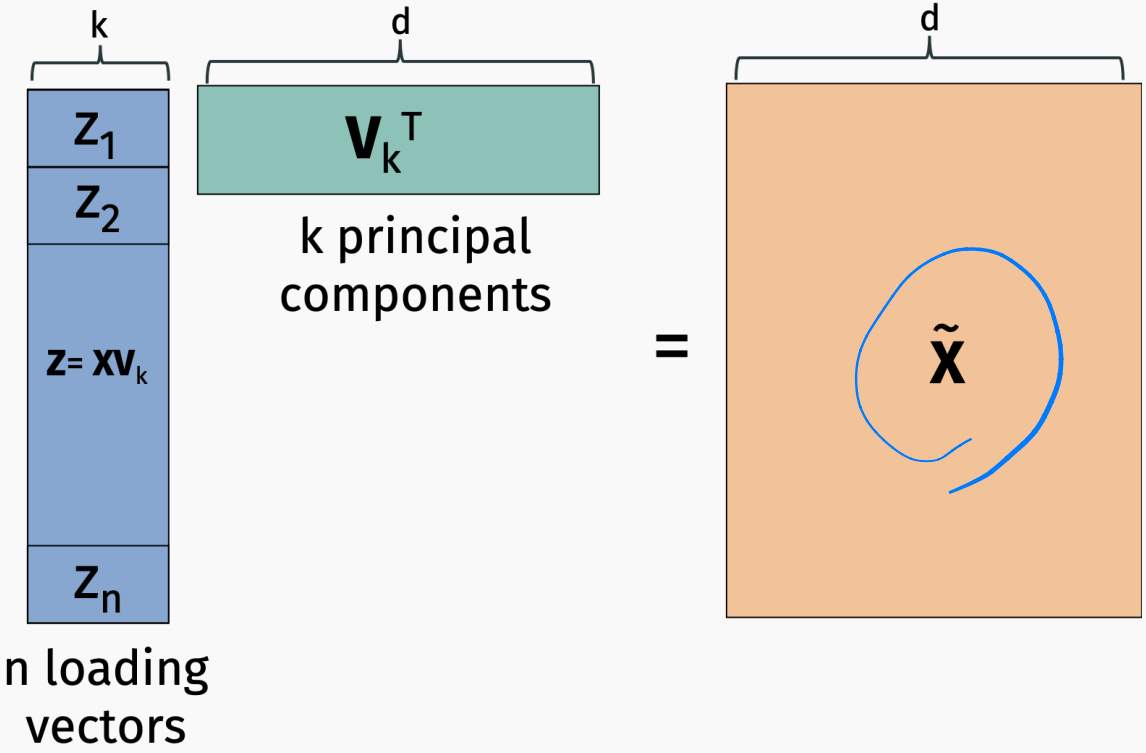
we know: $X = U \Sigma V^T$

$$V^T V_k = \begin{bmatrix} 1 & 0 & \\ 0 & \ddots & 1 \\ \hline & 0 & \end{bmatrix}_{d \times k} \Rightarrow \Sigma V^T V_k = \begin{bmatrix} \sigma_1 & & 0 \\ & \sigma_2 & \\ 0 & & \ddots \sigma_d \end{bmatrix}_{d \times d} \begin{bmatrix} 1 & 0 & \\ 0 & \ddots & 1 \\ \hline & 0 & \end{bmatrix}_{d \times k} = \begin{bmatrix} \sigma_1 & 0 & \\ 0 & \ddots & \sigma_k \\ \hline & 0 & \end{bmatrix}_{d \times k}$$

$$U \Sigma V^T V_k = \begin{bmatrix} u_1 & u_2 & \cdots & u_d \end{bmatrix}_{n \times d} \begin{bmatrix} \sigma_1 & & 0 \\ & \ddots & \\ 0 & & \sigma_k \\ \hline & 0 & \end{bmatrix}_{d \times k} = \underbrace{\begin{bmatrix} \sigma_1 u_1 & \sigma_2 u_2 & \cdots & \sigma_k u_k \end{bmatrix}_{n \times k}}_{\text{this is equal to } U_k \Sigma_k}$$

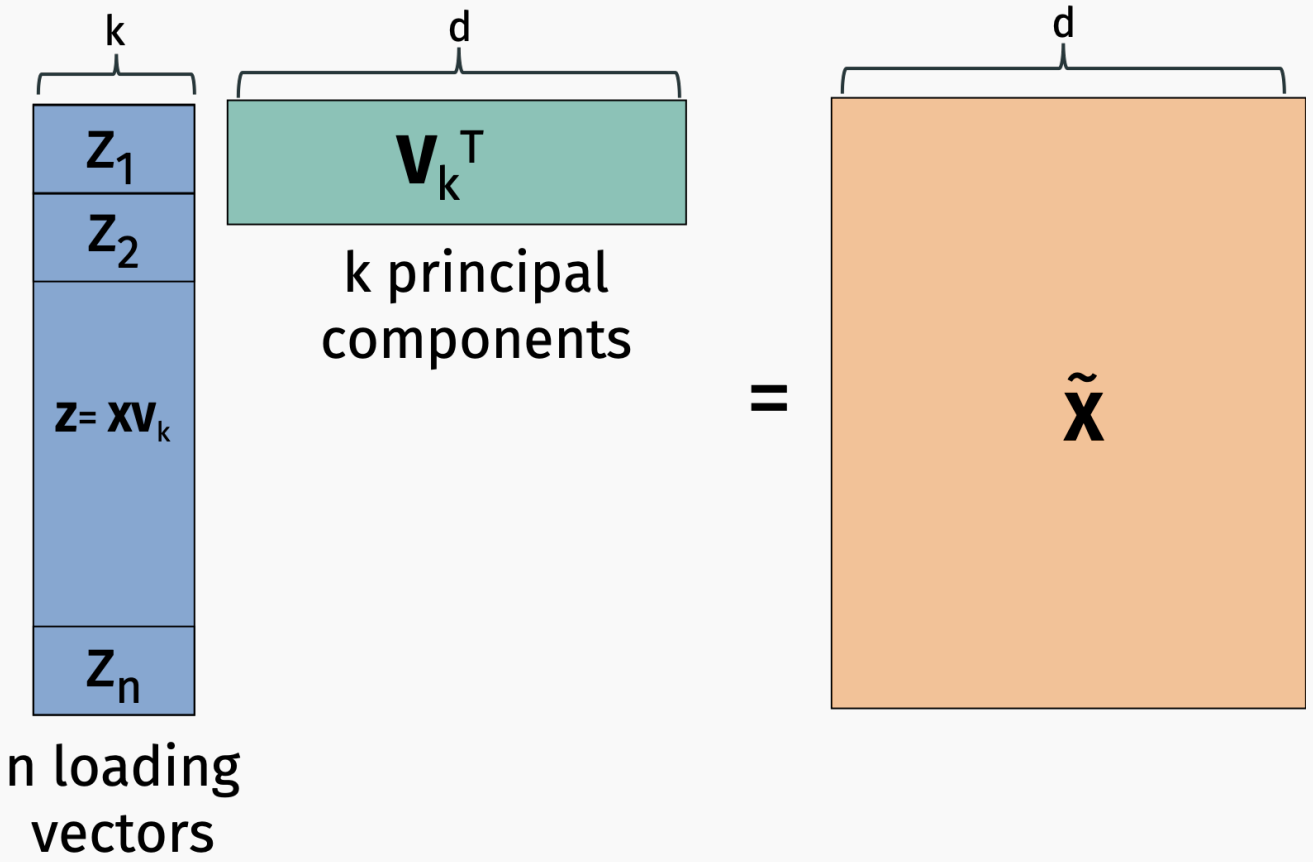
So for a model with $k$ hidden variables, we obtain an optimal autoencoder by setting $\mathbf{W}_1 = \mathbf{V}_k$, $\mathbf{W}_2 = \mathbf{V}_k^T$. $f(\mathbf{x}) = \mathbf{x}\mathbf{V}_k\mathbf{V}_k^T$.

# Principal Component Analysis




**Eckart–Young–Mirsky Theorem:** $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{V}_k\mathbf{V}_k^T$ is the optimal low-rank approximation to $\mathbf{X}$. So $\mathbf{W}_1 = \mathbf{V}_k$ and $\mathbf{W}_2 = \mathbf{V}_k^T$ are optimal autoencoder parameters.

# Principal Component Analysis (PCA)




Usually **x**'s columns (features) are mean centered and normalized to variance 1 before computing principal components.

# Singular value decomposition

**Computing the SVD.**

- Full SVD:

  `U,S,V = scipy.linalg.svd(X)`.

  Runs in $O(nd^2)$ time.

- Just the top $k$ components:

  `U,S,V = scipy.sparse.linalg.svds(X, k)`.

  Runs in roughly $O(ndk)$ time.

# Connection to eigen-decomposition

Recall that for a matrix $\mathbf{M} \in \mathbb{R}^{p \times p}$, $\mathbf{q}$ is an eigenvector of $\mathbf{M}$ if $\lambda \mathbf{q} = \mathbf{M}\mathbf{q}$ for a scalar $\lambda$.

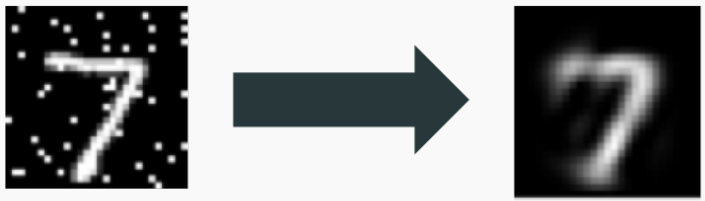
- $\mathbf{U}$'s columns (the left singular vectors) are the orthonormal eigenvectors of $\mathbf{X}\mathbf{X}^T$.
- $\mathbf{V}$'s columns (the right singular vectors) are the orthonormal eigenvectors of $\mathbf{X}^T\mathbf{X}$.
- $\sigma_i^2 = \lambda_i(\mathbf{X}\mathbf{X}^T) = \lambda_i(\mathbf{X}^T\mathbf{X})$

**Exercise:** Verify this directly. This means you can use any eigensolver for computing the SVD.

# PCA applications

**Like any autoencoder, PCA can be used for:**

- Feature extraction
- Denoising and rectification
- Data generation
- Compression
- Visualization



denoising

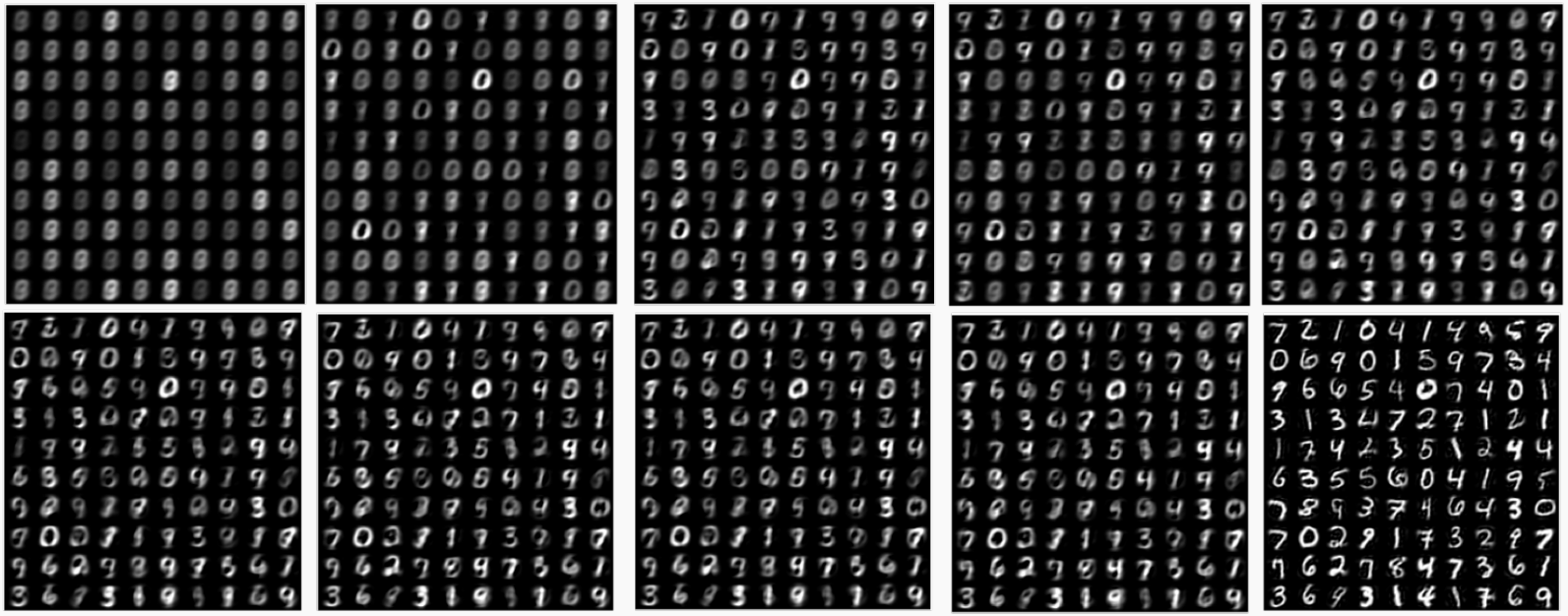
synthetic data generation

# Low-rank approximation

The larger we set $k$, the better approximation we get.

original data

rank 1 approx. | rank 2 approx. | rank 3 approx. | rank 4 approx. | rank 5 approx.

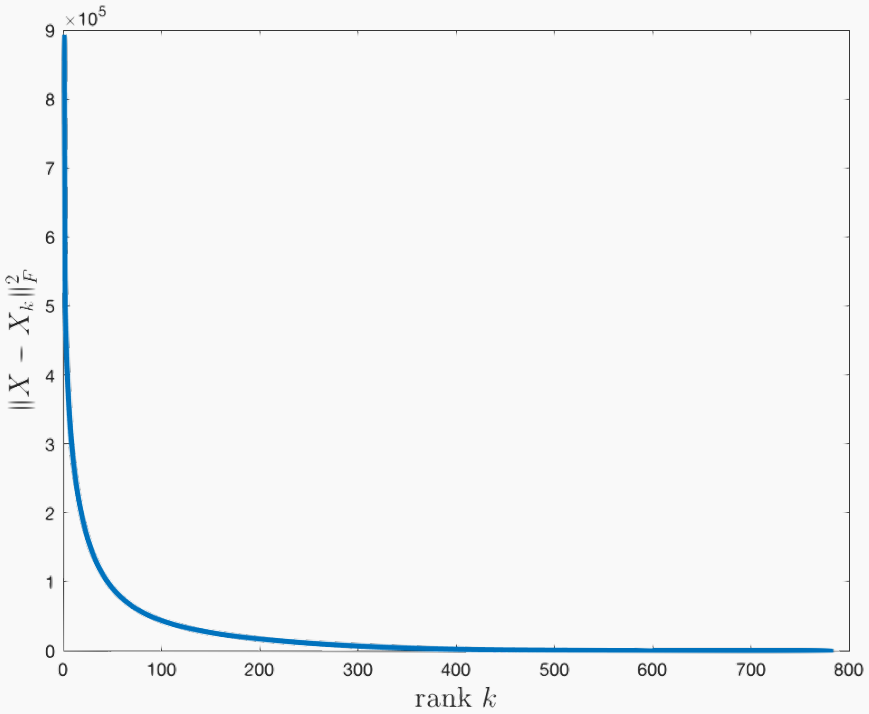


rank 6 approx. | rank 7 approx. | rank 8 approx. | rank 9 approx. | rank 50 approx.

# Low rank approximation

Error vs. $k$ is dictated by $\mathbf{X}$'s singular values. The singular values are often called the **spectrum** of $\mathbf{X}$.

$$\|\mathbf{X} - \mathbf{X}_k\|_F^2 = \sum_{i=k+1}^{d} \sigma_i^2.$$



# Column redundancy

**Colinearity** of data features leads to an approximately low-rank data matrix.

| | bedrooms | bathrooms | sq.ft. | floors | list price | sale price |
|---|---|---|---|---|---|---|
| home 1 | 2 | 2 | 1800 | 2 | 200,000 | 195,000 |
| home 2 | 4 | 2.5 | 2700 | 1 | 300,000 | 310,000 |
| . | . | . | . | . | . | . |
| . | . | . | . | . | . | . |
| . | . | . | . | . | . | . |
| home n | 5 | 3.5 | 3600 | 3 | 450,000 | 450,000 |

sale price $\approx 1.05 \cdot$ list price.

property tax $\approx .01 \cdot$ list price.

# Column redundancy

Sometimes these relationships are simple, other times more complex. But as long as there exists linear relationships between features, we will have a lower rank matrix.

$$\textbf{yard size} \approx \textbf{lot size} - \frac{1}{2} \cdot \textbf{square footage}.$$

$$\begin{aligned}\textbf{cumulative GPA} \approx & \frac{1}{4} \cdot \textbf{year 1 GPA} + \frac{1}{4} \cdot \textbf{year 2 GPA} \\ & + \frac{1}{4} \cdot \textbf{year 3 GPA} + \frac{1}{4} \cdot \textbf{year 4 GPA}.\end{aligned}$$

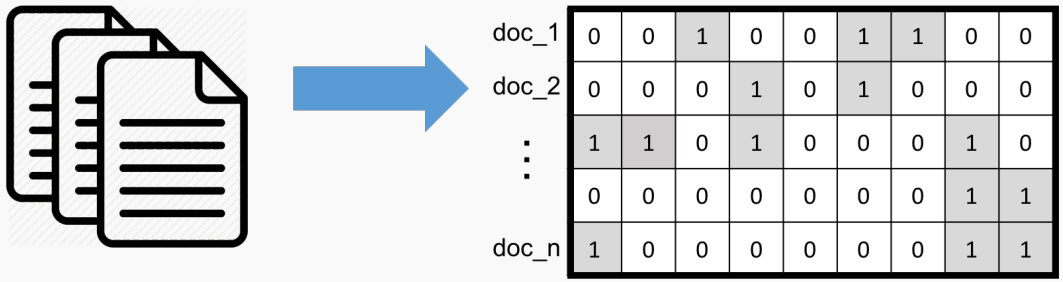
# Low-rank intuition

Two other examples of data with good low-rank approximations:

1. Genetic data:

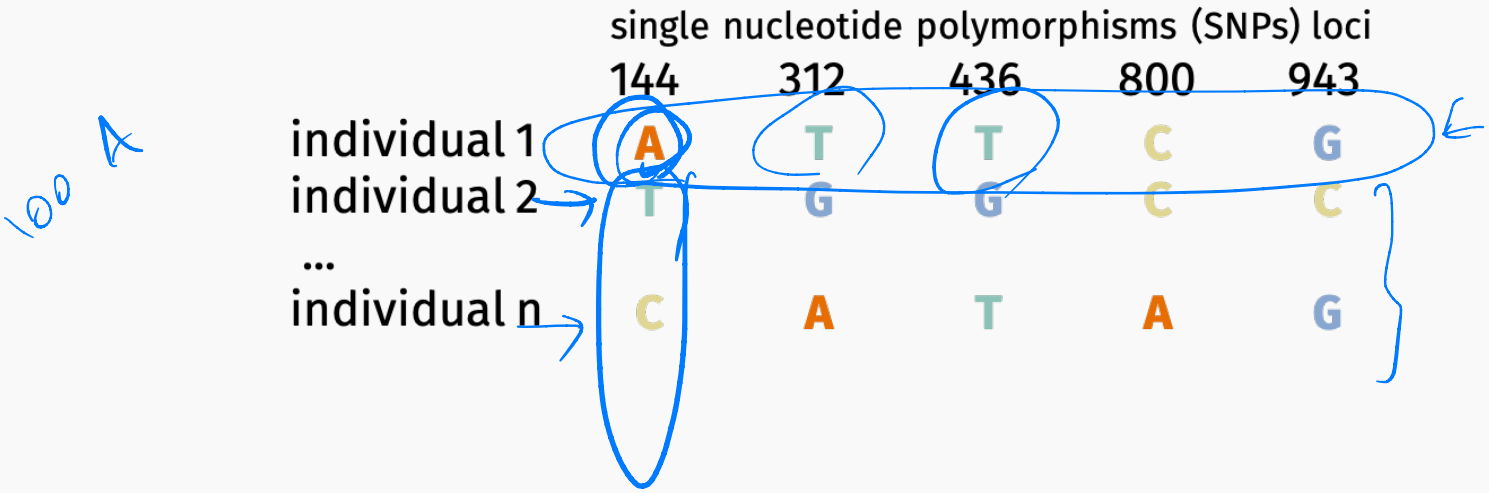
| | single nucleotide polymorphisms (SNPs) loci | | | | |
|---|---|---|---|---|---|
| | 144 | 312 | 436 | 800 | 943 |
| individual 1 | A | T | T | C | G |
| individual 2 | T | G | G | C | C |
| ... | | | | | |
| individual n | C | A | T | A | G |

2. "Term-document" matrix with bag-of-words data:

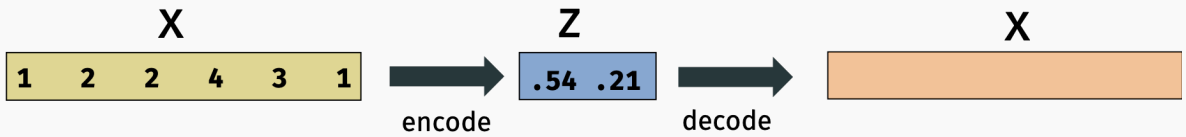


# Examples of low-rank structure

SNPs matrices tend to be very low-rank.

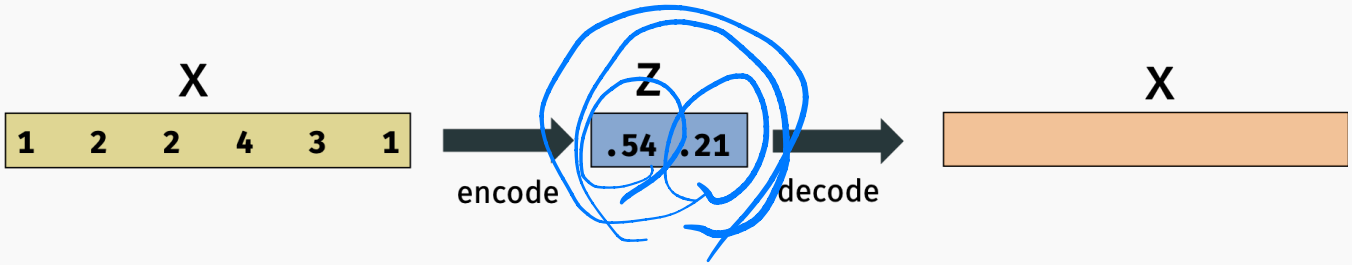



Most of the information in **x** is explained by just a few **latent variable**.



# Examples of low-rank structure

"Genes Mirror Geography Within Europe" – Nature, 2008.




In data collected from European populations, latent variables capture information about geography.

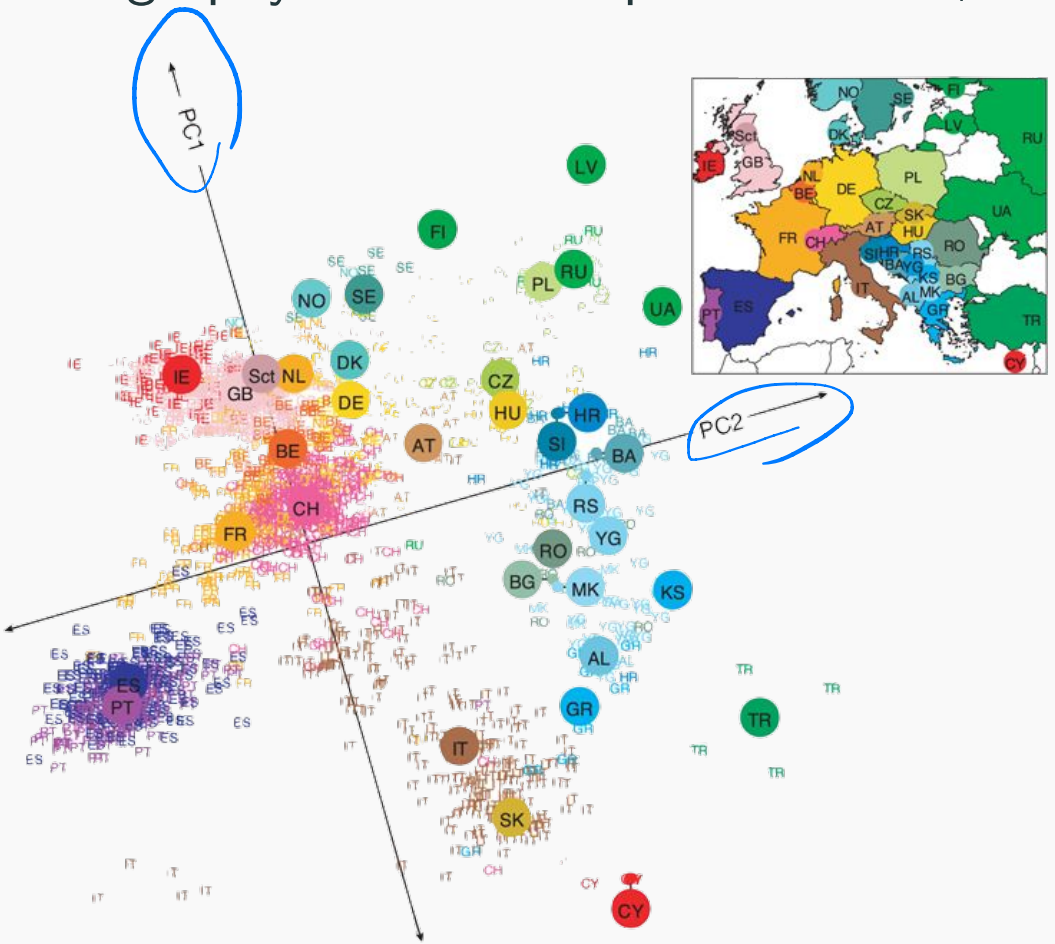
$\mathbf{z}[1] \approx$ relative north-south position of birth place

$\mathbf{z}[2] \approx$ relative east-west position of birth place

**Individuals born in similar places tend to have similar genes.**

# PCA for data visualization

"Genes Mirror Geography Within Europe" – Nature, 2008.



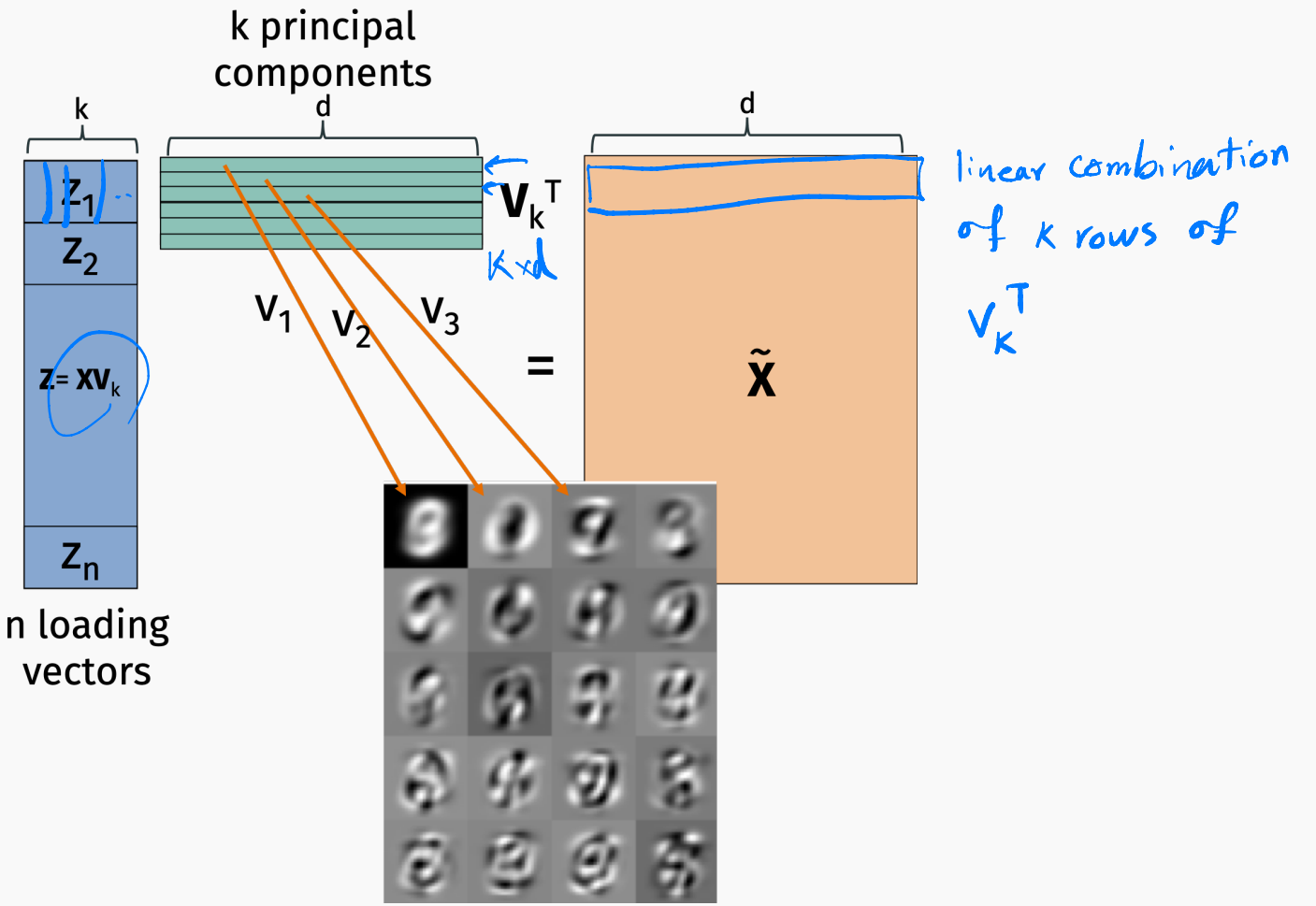

Genetic data can be nicely visualized using PCA! Plot each data example $\mathbf{x}$ using two loading variables in $\mathbf{z}$.

# Principal components

For more complex data, what do principal components and loading vectors look like?

# Principal components

MNIST **principal components**:

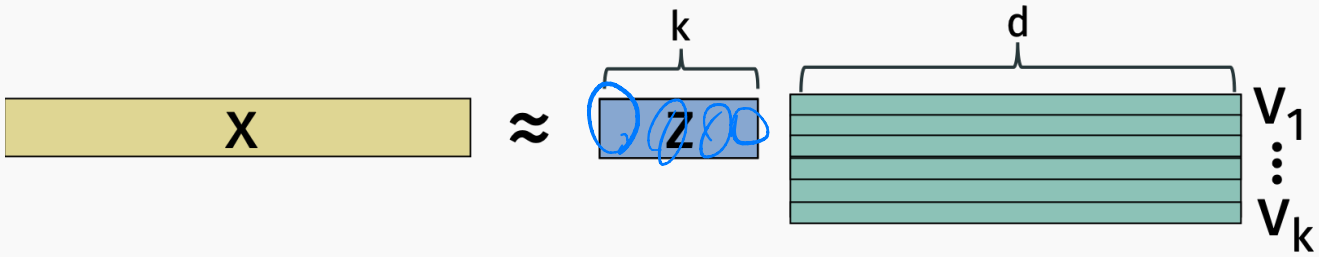


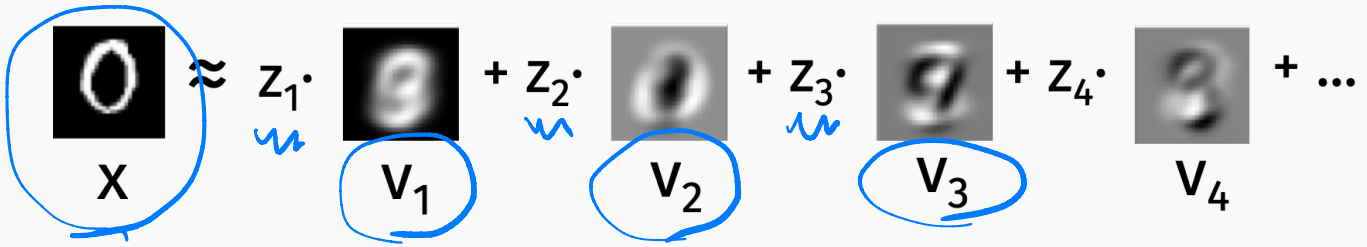

Principal components are a small set of vectors that can be recombined to approximate rows in $\tilde{\mathbf{X}}$.

# Loading vectors

What do the **loading vectors** looks like?

The loading vector $\mathbf{z}$ for an example $\mathbf{x}$ contains coefficients which recombine the top $k$ principal components $\mathbf{v}_1, \ldots, \mathbf{v}_k$ to approximately reconstruct $\mathbf{x}$.

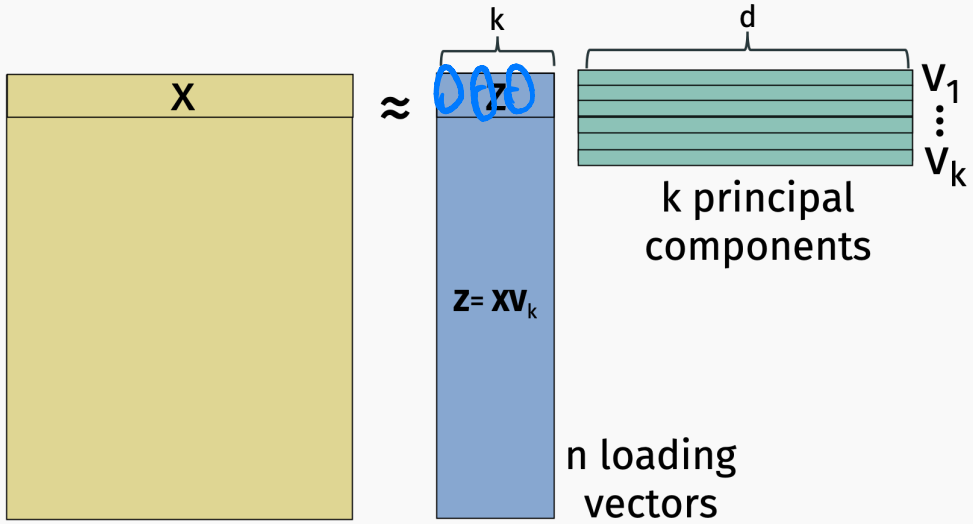


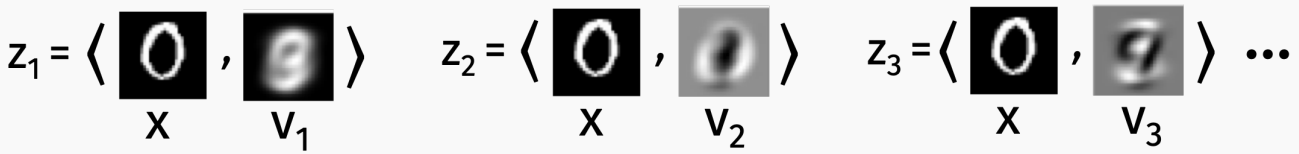




Provide a short "finger print" for any image $\mathbf{x}$ which can be used to reconstruct that image.

# Loading vectors: similarity view

For any $\mathbf{x}$ with loading vector $\mathbf{z}$, $\mathbf{z}_i$ is the inner product similarity between $\mathbf{x}$ and the $i^{\text{th}}$ principal component $\mathbf{v}_i$.

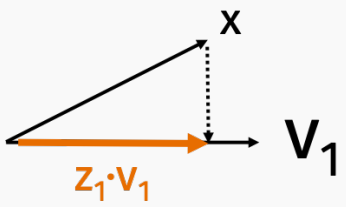


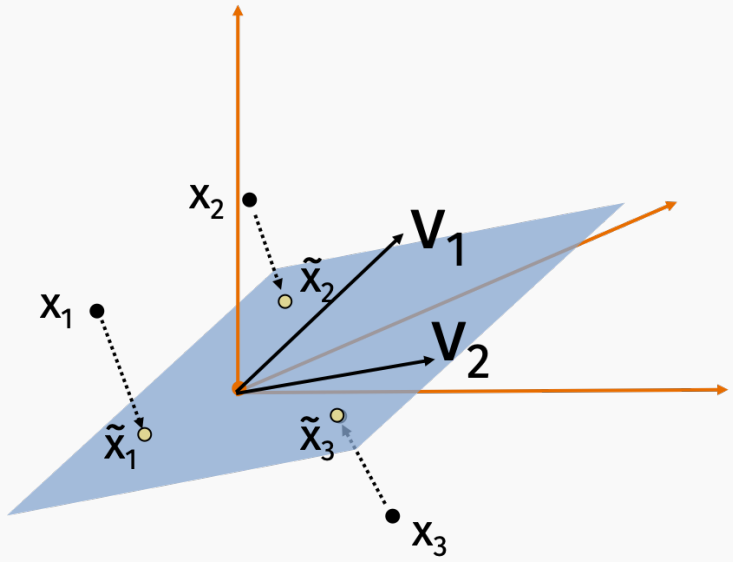



# Loading vectors: projection view

So we approximate $\mathbf{x} \approx \tilde{\mathbf{x}} = \langle \mathbf{x}, \mathbf{v}_1 \rangle \cdot \mathbf{v}_1 + \ldots + \langle \mathbf{x}, \mathbf{v}_k \rangle \cdot \mathbf{v}_k$.

$z_1$ $z_k$



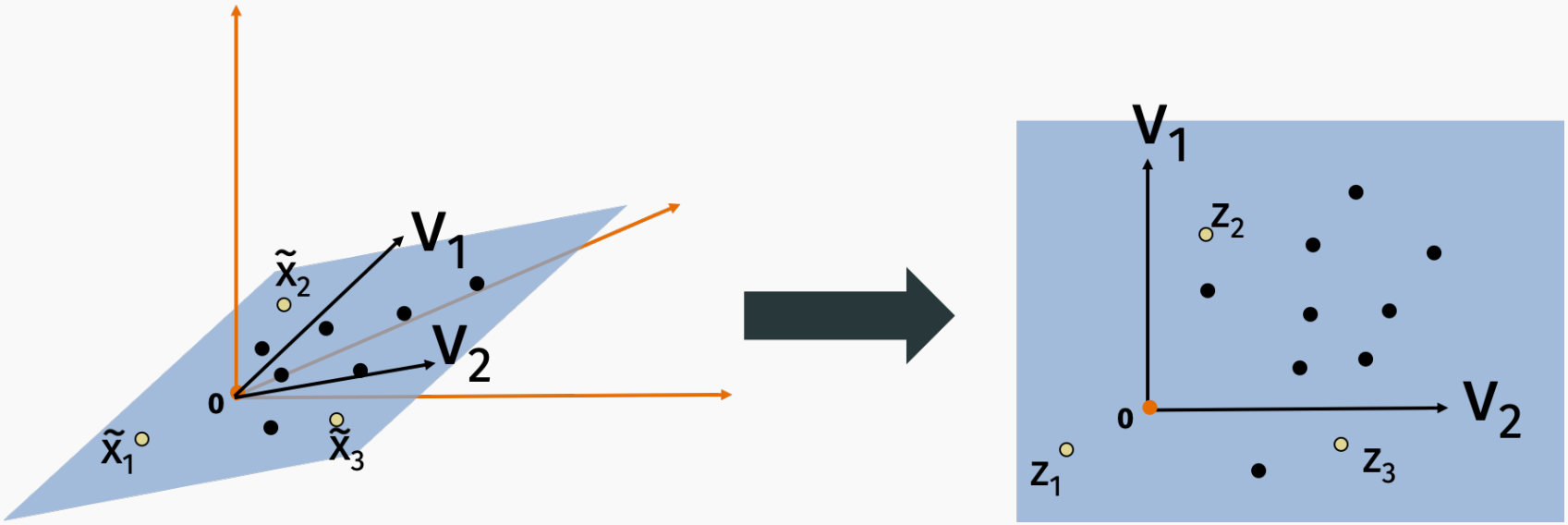




Since $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are orthonormal, this operation is a **projection** onto first $k$ principal components.

I.e. we are projecting $\mathbf{x}$ onto the $k$-dimensional subspace spanned by $\mathbf{v}_1, \ldots, \mathbf{v}_k$.

# Loading vectors: projection view

For an example $\mathbf{x}_i$, the loading vector $\mathbf{z}_i$ contains the coordinates in the projection space:



# Similarity preservation

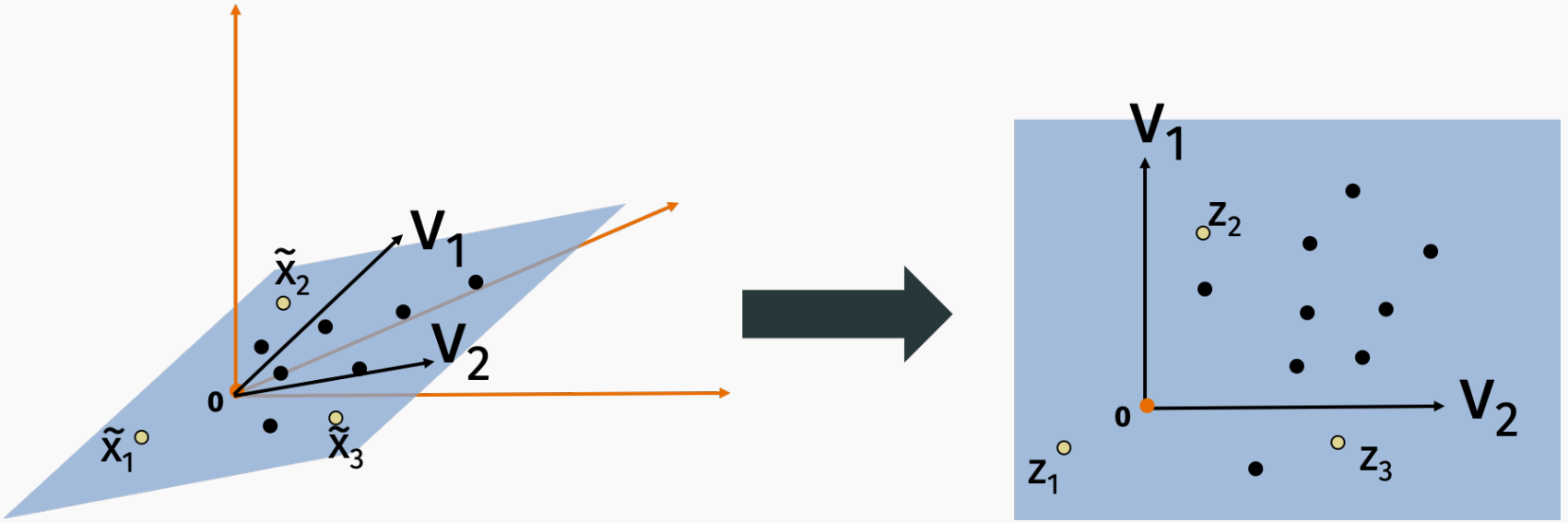
**Important takeaway for data visualization and more:** Latent feature vectors preserve similarity and distance information in the original data.

Let $\mathbf{x}_1 \ldots, \mathbf{x}_n \in \mathbb{R}^d$ be our original data vectors, $\mathbf{z}_1 \ldots, \mathbf{z}_n \in \mathbb{R}^k$ be our loading vectors (encoding), and $\tilde{\mathbf{x}}_1 \ldots, \tilde{\mathbf{x}}_n \in \mathbb{R}^d$ be our low-rank approximated data.

We have:

$$\|\tilde{\mathbf{x}}_i\|_2^2 = \|\mathbf{z}_i\|_2^2$$
$$\langle \tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j \rangle = \langle \mathbf{z}_i, \mathbf{z}_j \rangle$$
$$\|\tilde{\mathbf{x}}_i - \tilde{\mathbf{x}}_j\|_2^2 = \|\mathbf{z}_i - \mathbf{z}_j\|_2^2$$

# PCA preserves geometry of input data




$$\|\mathbf{x}_i\|_2^2 \approx \|\mathbf{z}_i\|_2^2$$

$$\langle \mathbf{x}_i, \mathbf{x}_j \rangle \approx \langle \mathbf{z}_i, \mathbf{z}_j \rangle$$

$$\|\mathbf{x}_i - \mathbf{x}_j\|_2^2 \approx \|\mathbf{z}_i - \mathbf{z}_j\|_2^2$$